

Contents

Fast & Slow Pointers — Complete Interview Handbook	1
Table of Contents	1
SECTION 1 — Pattern Overview	2
SECTION 2 — Recognition Guide	3
SECTION 3 — Decision Framework	3
SECTION 4 — Why This Pattern Works	4
SECTION 5 — Algorithm Framework	5
SECTION 6 — Time & Space Complexity	6
SECTION 7 — Edge Cases	6
SECTION 8 — Pros & Cons	7
SECTION 9 — Real World Applications	7
SECTION 10 — Interview Strategy	8
SECTION 11 — Pattern Variations	8
SECTION 12 — Comparison With Other Patterns	8
SECTION 13 — Problem Classification	9
SECTION 14 — 30 Interview Problems	9
SECTION 15 — Common Mistakes	12
SECTION 16 — Optimization Techniques	13
SECTION 17 — Multiple Language Templates	13
SECTION 18 — Dry Runs	14
SECTION 19 — Advanced Concepts	15
SECTION 20 — Senior Engineer Insights	15
SECTION 21 — Cheat Sheet	15
SECTION 22 — Revision Notes (5-Minute Review)	15
SECTION 23 — Practice Roadmap	16
SECTION 24 — Memory Tricks	16
SECTION 25 — Final Summary	16

Fast & Slow Pointers — Complete Interview Handbook

Pattern #3 of 28 | DSA Pattern Mastery Series Audience: Senior/Staff SWE interviews (Google, Amazon, Meta, Microsoft, Uber, Airbnb, Atlassian, Grab, Careem, Noon, Talabat, Dubai/Saudi & India Tier-1/Tier-2 product companies) **Companion:** Pairs with Striver's A2Z DSA Course — Linked List section

Table of Contents

1. Pattern Overview · 2. Recognition Guide · 3. Decision Framework · 4. Why This Pattern Works · 5. Algorithm Framework · 6. Time & Space Complexity · 7. Edge Cases · 8. Pros & Cons · 9. Real World Applications · 10. Interview Strategy · 11. Pattern Variations · 12. Comparison With Other Patterns · 13. Problem Classification · 14. 30 Interview Problems · 15. Common Mistakes · 16. Optimization Techniques · 17. Multiple Language Templates · 18. Dry Runs · 19. Advanced Concepts · 20. Senior Engineer Insights · 21. Cheat Sheet · 22. Revision Notes · 23. Practice Roadmap · 24. Memory Tricks · 25. Final Summary

SECTION 1 — Pattern Overview

1.1 What Is This Pattern?

Also called **Floyd's Cycle Detection** or the "Tortoise and Hare" algorithm, Fast & Slow Pointers uses two pointers traversing the same sequence (usually a linked list, sometimes an array treated as a functional graph) at **different speeds** — typically slow moves 1 step, fast moves 2 steps per iteration. The relative speed difference lets you detect cycles, find midpoints, or detect specific offsets **without extra memory**.

1.2 Why Was This Pattern Invented?

Detecting a cycle in a linked list naively requires storing every visited node in a hash set — $O(n)$ space. Robert Floyd's insight (1967) was that **if a cycle exists, a faster-moving pointer must eventually "lap" a slower one inside the cycle**, exactly like two runners on a circular track — meaning you can detect the cycle in $O(1)$ space by simply checking if the two pointers ever meet.

1.3 Real Intuition Behind The Pattern

On a circular running track, a faster runner will eventually catch up to and pass a slower runner, no matter where they started, as long as the track is a loop. If the track were a straight, non-looping road, the faster runner would just finish first and never "meet" the slower one again. This binary outcome — **meet $\Rightarrow$ cycle exists; reach the end $\Rightarrow$ no cycle** — is the entire basis of the pattern.

1.4 Mental Model

Think of `slow` as marking "elapsed distance d " and `fast` as marking "elapsed distance $2d$." If they're ever at the same node, the gap between their distances (d) must be a multiple of the cycle length — a purely arithmetic consequence of modular arithmetic on a cycle.

1.5 Visual Explanation

```
List:  1 → 2 → 3 → 4 → 5
          ↑       ↓
        8 ← 7 ← 6
```

```
slow: 1 → 2 → 3 → 4 → 5 → 6 → 7 ...
fast:  1 → 3 → 5 → 7 → 4 → 6 → 8 ...
```

Eventually `slow` and `fast` land on the SAME node inside the loop (3→4→5→6→7→8→3...) - cycle confirmed.

1.6 Simple Analogy

Two friends jogging on a circular park track, one twice as fast as the other — if the track loops, the faster friend eventually laps the slower one and they high-five mid-lap. If it's a straight path with an end, the faster friend just finishes and waits at the exit — no meeting happens mid-path.

1.7 When Should I Immediately Think About Using This Pattern?

- Linked list problems mentioning **"cycle," "loop," "starts of the cycle."**
- "Find the **middle** of a linked list" in a single pass.
- "Detect a duplicate number" in an array where values point to indices (functional graph cycle).
- "Determine if a sequence eventually repeats" (e.g., Happy Number problem).

SECTION 2 — Recognition Guide

2.1 Keywords In Problem Statement

Keyword	Signal
"cycle" / "loop"	Direct signal for Floyd's algorithm
"middle of the linked list"	Fast/slow with fast moving 2x
"starts of a cycle"	Floyd's algorithm phase 2 (finding cycle entry)
"happy number" / "eventually repeats"	Cycle detection over a functional sequence, not a literal linked list
"duplicate number, values in range [1,n]"	Array-as-linked-list cycle detection
"without extra space"	Rules out hash-set based visited tracking, strongly implies Fast & Slow

2.2 Hidden Hints

A problem defined over a **sequence generated by repeatedly applying a function** ($x = f(x)$) — even if it doesn't mention linked lists at all — is very often solvable via Fast & Slow (e.g., LeetCode's Happy Number, Find the Duplicate Number).

2.3 Interview Clues

Interviewer explicitly rules out extra space ("can you do it in $O(1)$ space?") after you propose a hash-set solution — the strongest possible nudge toward Fast & Slow.

2.4 Common Trick Words

"Detect if," "determine the entry point of," "kth node from the end" (a fast/slow variant with a fixed offset gap rather than 2x speed).

2.5 What Interviewers Expect

Correct implementation of both phases of Floyd's algorithm (cycle **detection**, then cycle **entry point**), and the mathematical justification for why the entry-point-finding phase works (not just "it's a known trick").

2.6 When NOT To Use This Pattern

- You need **random access** or need to examine all elements (not just detect a structural property) — arrays with direct indexing may not need this trick at all.
- The structure is **not a chain/sequence with a single "next" pointer** (e.g., trees with multiple children, general graphs) — use standard graph cycle detection (DFS with colors / Union-Find) instead.
- You need the **actual set of visited nodes**, not just cycle existence — then $O(n)$ space hashing might be simpler and equally acceptable if space isn't constrained.

SECTION 3 — Decision Framework

Is the structure a single-successor chain (linked list, or array-as-function $f(x)$)?

Yes

No → use Graph Cycle Detection (DFS colors / Union-Find)

Do you need $O(1)$ space?

Yes No

FAST & SLOW Hash Set of visited nodes is simpler and
POINTERS acceptable if $O(n)$ space is fine

Do you need the CYCLE ENTRY POINT (not just yes/no)?

Yes → after meeting, reset one pointer to head, move both
 at speed 1 - they meet exactly at the cycle start

No → simply report "cycle detected" once pointers meet

Why: Fast & Slow is strictly about $O(1)$ space cycle/midpoint detection over chain-like structures; anything requiring full traversal history or multi-branch structures needs a different tool.

SECTION 4 — Why This Pattern Works

4.1 Mathematical Proof of Meeting (Cycle Detection)

Let the list have a non-cyclic prefix of length l and a cycle of length c . After t steps, *slow* is at position $l + ((t - l) \bmod c)$ once inside the cycle, and *fast* is at position equivalent to $2t$. They meet when $2t - t \equiv l \pmod c$ for $t \geq l$, i.e., $t \equiv l \pmod c$. Since t increases by 1 each step, some multiple of c greater than or equal to l will always be reached in finite steps — **guaranteeing a meeting point exists if and only if a cycle exists.**

4.2 Proof of Entry-Point-Finding Phase

At the meeting point, *slow* has traveled $l + a$ steps (where a is distance traveled inside the cycle) and *fast* has traveled $2(l + a)$ steps. Since *fast* has looped around some integer number of times k , we get $2(l + a) - (l + a) = k \cdot c$, i.e., $l + a = k \cdot c$, so $l \equiv -a \pmod c$, i.e., $l \equiv (c - a) \pmod c$. This means: if you place one pointer back at the **head** (distance l from cycle start) and leave the other at the **meeting point** (distance $c - a$ from cycle start, equivalently), and move both one step at a time, **they will meet exactly at the cycle's entry node** after c steps. This is a beautiful, non-obvious modular arithmetic result — memorize the *proof sketch*, not just the trick.

4.3 Logical/Intuitive Argument

Two runners on a track, one twice as fast: the gap between them changes by exactly 1 lap-distance-unit each cycle iteration once both are on the loop, so the gap must eventually become 0 (a full lap difference) — they must meet.

4.4 Correctness of “Find the Middle” Variant

When *fast* reaches the end (null or one before), *slow* has traveled exactly half the distance, because *fast* always travels exactly twice as far in the same number of steps — a direct, non-modular consequence of the 2x speed ratio, no cycle needed.

SECTION 5 — Algorithm Framework

5.1 Step-by-Step Framework (Cycle Detection + Entry Point)

1. Initialize `slow = head`, `fast = head`.
2. Loop while `fast` and `fast.next` are non-null: `slow = slow.next`; `fast = fast.next.next`.
3. If `slow == fast` at any point, a cycle exists — break out to phase 2.
4. If the loop exits naturally (`fast` or `fast.next` becomes null), no cycle exists.
5. **Phase 2 (entry point):** reset `slow2 = head`; move `slow` and `slow2` one step at a time until `slow == slow2` — that node is the cycle's entry point.

5.2 General Template (Language-Independent Pseudocode)

```
function hasCycle(head):
    slow = head
    fast = head
    while fast != null and fast.next != null:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            return true
    return false

function findCycleStart(head):
    slow = head
    fast = head
    while fast != null and fast.next != null:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            ptr = head
            while ptr != slow:
                ptr = ptr.next
                slow = slow.next
            return ptr          # cycle entry node
    return null                # no cycle
```

5.3 Find-Middle Variant

```
function findMiddle(head):
    slow = head
    fast = head
    while fast != null and fast.next != null:
        slow = slow.next
        fast = fast.next.next
    return slow                # middle node (upper-middle if even length)
```

5.4 Interview Thinking Process

1. "This is a chain-structured traversal — I'll use two pointers at different speeds to avoid $O(n)$ extra space."
2. "If a cycle exists, the fast pointer will eventually lap the slow one — I can prove this via modular arithmetic on the cycle length."
3. "If I also need the cycle's entry point, I'll reset one pointer to head and advance both at equal speed — they'll meet exactly at the entry, which I can justify with the $\mu \equiv -a \pmod{\lambda}$ relation."
4. "Let me dry-run on a small list with a known cycle to validate indices before finalizing."

SECTION 6 — Time & Space Complexity

Case	Time	Space	Why
Worst Case	$O(n)$	$O(1)$	Fast pointer traverses at most $2n$ nodes before either exiting or meeting slow
Average Case	$O(n)$	$O(1)$	Independent of cycle position — bounded by list length + cycle length
Best Case	$O(1)$ (immediate cycle at head) to $O(n)$	$O(1)$	Still requires at least one comparison
Amortized	$O(n)$ total across both phases (detection + entry-finding)	$O(1)$	Phase 2 adds at most another $O(n)$ traversal, still linear overall

SECTION 7 — Edge Cases

Edge Case	Example	Handling
Empty list	<code>head = null</code>	Immediately return <code>false/null</code>
Single node, no self-loop	<code>[5] → null</code>	Loop condition fails immediately, no cycle
Single node with self-loop	<code>[5] → itself</code>	<code>fast</code> and <code>slow</code> both point to node 5 immediately on next iteration — cycle detected correctly
Two nodes forming a cycle	<code>1 → 2 → 1</code>	Meeting occurs within 2 iterations — verify with dry run
Cycle length 1 vs list length 1,000,000	huge non-cyclic prefix	Still $O(n)$ — proof in §4.1 doesn't depend on cycle length being small
No cycle, even-length list	<code>[1, 2, 3, 4]</code>	<code>fast</code> reaches null cleanly, loop exits, no false positive
No cycle, odd-length list	<code>[1, 2, 3]</code>	<code>fast.next</code> becomes null exactly when expected, exits safely
Middle-of-list with even length	<code>[1, 2, 3, 4]</code>	Convention: <code>fast/slow</code> lands on second-middle element (index 2, value 3) — clarify with interviewer which “middle” is expected

Common mistakes: checking `fast != null` but forgetting `fast.next != null` (causes null pointer dereference on `fast.next.next`); assuming a cycle always starts at the head (rarely true — must run phase 2 to find the actual entry).

SECTION 8 — Pros & Cons

Advantages: $O(1)$ space cycle detection (versus $O(n)$ with a hash set); simultaneously finds mid-points without needing to know the list length in advance; elegant, provably correct via clean modular arithmetic. **Disadvantages:** Only works on **single-successor chain structures**; the entry-point-finding phase, while correct, is non-obvious and easy to misremember without understanding the proof. **Trade-offs:** Fast & Slow ($O(1)$ space) vs Hash Set ($O(n)$ space, arguably more intuitive to explain) — prefer Fast & Slow when space is constrained or when the interviewer explicitly asks for $O(1)$ space. **Limitations:** Doesn't generalize to graphs with multiple outgoing edges per node — that requires DFS-based cycle detection with color marking. **Inefficient when:** N/A for its core use case — it is already asymptotically optimal for chain-based cycle detection; there's no faster known approach for this exact problem shape.

SECTION 9 — Real World Applications

Domain	Application
Operating Systems	Detecting cycles in process wait-for graphs (deadlock detection) — chain-like resource allocation graphs use similar cycle-detection logic
Databases	Detecting circular references in linked record structures (e.g., circular foreign key chains)
Compilers	Detecting infinite loops in simplified control-flow chains during static analysis
Networking	Detecting routing loops in simplified single-next-hop routing tables
Memory Management	Garbage collectors use cycle-detection-like reasoning (though typically via mark-sweep, not literally Floyd's) to identify reference cycles that would otherwise leak memory in naive refcounting systems
Blockchain	Detecting malformed circular chains in linked block references (data integrity checks)
Distributed Systems	Detecting cyclic dependency chains in service call graphs (a simplified same-idea generalization)
Google/Amazon/Meta interview pipelines	Frequently used as a canonical $O(1)$ -space linked-list warm-up question to test fundamentals before moving to harder graph problems

SECTION 10 — Interview Strategy

How seniors answer: They state the $O(1)$ space constraint proactively, explain the “runners on a track” intuition before coding, and — critically — can derive the entry-point-finding phase’s correctness on the whiteboard using the $-a \pmod{}$ relationship rather than reciting it as a memorized trick.

How juniors answer: They often only know “move slow by 1, fast by 2, if they meet there’s a cycle” but cannot explain *why* resetting one pointer to head and moving both at speed 1 finds the entry point — a dead giveaway under a “why does that work?” follow-up.

Typical follow-ups: “Can you find the length of the cycle?” (once inside the cycle, keep one pointer fixed and advance the other until it returns — count steps). “Can you do this with $O(n)$ space instead — which is simpler to explain?” (hash set comparison). “How would this generalize to a graph with multiple next pointers?”

Optimization questions: “Is there a way to avoid the second traversal phase?” (No — it’s mathematically necessary; discuss why the meeting point alone doesn’t tell you the entry point directly).

SECTION 11 — Pattern Variations

Variation	Description	Example
Cycle Detection	Detect existence of a cycle	Linked List Cycle
Cycle Entry Point	Find where the cycle begins	Linked List Cycle II
Find Middle	Fast pointer 2x speed finds midpoint	Middle of the Linked List
Palindrome Check via Midpoint	Find middle, reverse second half, compare	Palindrome Linked List
Functional Graph Cycle (array-based)	Treat array values as “next pointers”	Find the Duplicate Number, Circular Array Loop
Sequence Cycle Detection (non-list)	Apply Floyd’s to a mathematically generated sequence	Happy Number
Fixed Offset Fast/Slow (not speed-based)	Fast starts k nodes ahead, both move at speed 1	Kth Node From End, Remove Nth Node From End

SECTION 12 — Comparison With Other Patterns

Pattern	Difference	When To Prefer
Two Pointers	Operates on sorted arrays for pair search, not chain traversal for structural detection	Static pair/triplet search
Hashing	Detects cycles via visited-set membership, $O(n)$ space, but conceptually simpler	Space isn’t constrained, want simpler code
DFS with Color Marking	General graph cycle detection for multi-successor structures	Trees/graphs with branching, not simple chains

Pattern	Difference	When To Prefer
Union-Find	Detects cycles in undirected graphs during edge processing	Building a graph incrementally and checking cycles as edges are added

Comparison Table

Aspect	Fast & Slow Pointers	Hash Set Cycle Detection	DFS Cycle Detection (Graphs)
Space	O(1)	O(n)	O(V) for visited/color arrays
Structure	Single-successor chain	Any structure	General graphs (multi-successor)
Finds entry point	Yes (phase 2)	Yes (first repeated node)	N/A (different problem framing)
Complexity to explain	Higher (non-obvious proof)	Lower (intuitive)	Moderate

SECTION 13 — Problem Classification

Difficulty	Characteristics	Examples
Easy	Simple cycle detection or middle-finding	Linked List Cycle, Middle of the Linked List
Medium	Entry point finding, palindrome check via midpoint	Linked List Cycle II, Palindrome Linked List, Remove Nth Node From End
Hard	Array-as-function cycle detection with constraints	Find the Duplicate Number, Circular Array Loop
Very Hard	Combined with additional invariants or multiple simultaneous constraints	Reorder List (midpoint + reverse + merge combination), complex multi-phase list restructuring

SECTION 14 — 30 Interview Problems

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
1	Linked List Cycle	Easy	Amazon, Meta, Microsoft, Google	Direct Floyd's cycle detection	Foundational cycle detection
2	Linked List Cycle II	Medium	Amazon, Meta, Google	Cycle entry-point phase 2	Modular arithmetic proof application

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
3	Middle of the Linked List	Easy	Amazon, Microsoft	Fast 2x speed to find midpoint	Basic fast/slow mechanics
4	Palindrome Linked List	Medium	Meta, Amazon, Microsoft	Midpoint + reversal + comparison	Multi-step combination
5	Remove Nth Node From End of List	Medium	Amazon, Meta, Microsoft	Fixed-offset fast/slow variant	Offset-based two-pointer
6	Reorder List	Medium	Amazon, Meta	Midpoint + reverse + merge	Complex multi-phase pattern combination
7	Happy Number	Easy	Amazon, Google	Cycle detection over a generated sequence	Non-list cycle detection
8	Find the Duplicate Number	Medium	Amazon, Meta, Google, Microsoft	Array-as-function cycle detection	Array-as-linked-list mental model
9	Circular Array Loop	Medium	Google	Cycle detection with directional constraints	Constrained cycle detection
10	Intersection of Two Linked Lists	Easy	Amazon, Microsoft, Meta	Related pointer-alignment technique (not strictly Floyd's but same family)	Pointer alignment via length difference
11	Split Linked List in Parts	Medium	Google	Related fast-traversal length calculation	Length-aware traversal
12	Rotate List	Medium	Amazon, Microsoft	Fast traversal to find length + rotation point	Length + pointer manipulation
13	Delete the Middle Node of a Linked List	Medium	Amazon	Fast/slow midpoint with predecessor tracking	Midpoint + deletion
14	Sort List	Medium	Microsoft, Amazon	Midpoint finding as part of merge sort on linked list	Cross-pattern (Fast/Slow + Merge Sort)

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
15	Reverse Nodes in Even Length Groups	Hard	Google	Fast traversal with grouped counting	Advanced grouped traversal
16	Linked List Random Node (Reservoir Sampling context)	Medium	Amazon	Related traversal fundamentals	Traversal + probability (tangential)
17	Add Two Numbers II	Medium	Amazon, Meta	Related but uses stack; good contrast problem	Recognizing when Fast/Slow does NOT apply
18	Odd Even Linked List	Medium	Amazon, Microsoft	Related pointer partitioning on a single chain	Chain partitioning without extra space
19	Swap Nodes in Pairs	Medium	Amazon, Meta	Related same-direction pointer manipulation	Pairwise pointer swapping
20	Palindrome Number (non-list variant, contrast)	Easy	Amazon	Contrast: solved without pointers on a list	Pattern-boundary awareness
21	Cycle Length in a Linked List (variant)	Medium	Google	Post-detection cycle length computation	Extending Floyd's phase 2
22	Detect and Remove Loop in Linked List	Medium	Amazon, Microsoft	Detection + entry point + removal	Full end-to-end Floyd's application
23	Find the Start of a Loop (variant framing)	Medium	Amazon	Entry-point proof application	Reinforce $\mu \equiv -a \pmod{\lambda}$ proof
24	Convert Sorted List to Binary Search Tree	Medium	Amazon, Microsoft	Midpoint finding used to build balanced BST	Cross-pattern (Fast/Slow + Tree construction)
25	Linked List Components	Medium	Google	Traversal fundamentals combined with set membership	Traversal + hashing hybrid

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
26	Flatten a Multilevel Doubly Linked List	Medium	Meta, Microsoft	Related traversal (not Floyd's but same data structure family)	Recognizing structural differences
27	Copy List with Random Pointer	Medium	Amazon, Meta, Microsoft	Related traversal + hashing (contrast problem)	Pattern-boundary awareness
28	Linked List Cycle III (custom/interview variant: find kth node in cycle)	Hard	Google	Extension of entry-point + offset logic	Advanced modular arithmetic application
29	Design Circular Queue (conceptual link)	Medium	Amazon, Microsoft	Conceptual reinforcement of circular structure reasoning	Circular structure intuition
30	Detect Cycle in a Directed Graph (contrast, general graph)	Medium	Google, Meta	Contrast case requiring DFS coloring instead of Floyd's	Understanding pattern's structural limits

SECTION 15 — Common Mistakes

1. Checking only `fast != null` and forgetting `fast.next != null`, causing a null-pointer crash on `fast.next.next`. *Fix*: always guard both conditions in the while loop.
2. Assuming the cycle starts at the head. *Fix*: always run phase 2 (reset to head, advance both at speed 1) to find the true entry point.
3. Confusing “meeting point” with “entry point” — they are almost never the same node. *Fix*: internalize the proof from §4.2.
4. Forgetting edge cases like a single self-looping node or a two-node cycle. *Fix*: explicitly dry-run these before finalizing.
5. Misapplying Floyd's to multi-successor structures (trees, general graphs) where each node has more than one “next.” *Fix*: recognize the pattern only applies to single-successor chains.

Why people fail: they memorize “slow moves 1, fast moves 2” without understanding the modular arithmetic that guarantees a meeting occurs and separately guarantees the entry-point trick — under a “why” follow-up or a slightly modified variant (e.g., cycle length, kth node in cycle), memorization alone breaks down.

SECTION 16 — Optimization Techniques

- **Time:** No further asymptotic optimization exists — $O(n)$ is optimal for this problem shape; focus instead on minimizing constant-factor overhead (avoid redundant traversals).
 - **Space:** Already $O(1)$ — the entire point of the pattern versus hash-set alternatives.
 - **Readability:** Clearly name pointers *slow/fast* (never generic *p1/p2*); comment the “why” of the entry-point reset step, since it’s the least intuitive part.
 - **Interview performance:** Draw the cycle on the whiteboard with *slow* and *fast* labeled before deriving the entry-point logic — visual + algebraic proof together is the strongest signal of seniority.
-

SECTION 17 — Multiple Language Templates

Java

```
public boolean hasCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true;
    }
    return false;
}
```

JavaScript

```
function hasCycle(head) {
    let slow = head, fast = head;
    while (fast && fast.next) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow === fast) return true;
    }
    return false;
}
```

PHP

```
function hasCycle($head): bool {
    $slow = $head; $fast = $head;
    while ($fast !== null && $fast->next !== null) {
        $slow = $slow->next;
        $fast = $fast->next->next;
        if ($slow === $fast) return true;
    }
    return false;
}
```

Python

```
def has_cycle(head):
    slow, fast = head, head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            return True
    return False
```

Go

```
func hasCycle(head *ListNode) bool {
    slow, fast := head, head
    for fast != nil && fast.Next != nil {
        slow = slow.Next
        fast = fast.Next.Next
        if slow == fast {
            return true
        }
    }
    return false
}
```

C++

```
bool hasCycle(ListNode *head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}
```

SECTION 18 — Dry Runs

Small Input

1 → 2 → 3 → 4 → 2 (cycle back to node 2, i.e., $\mu=1$, $\lambda=3$)

slow=1, fast=1

step1: slow=2, fast=3

step2: slow=3, fast=2 (fast wrapped: 4→2)

step3: slow=4, fast=4 → MEET at node 4

Phase 2: ptr=head(1), slow stays at 4

ptr=1, slow=4 → not equal → ptr=2, slow=2 → EQUAL → entry point = node 2 (matches =1 prefix: node 1 →

Large Input (Conceptual)

For a list of 100,000 nodes with a cycle of length 500 starting at node 99,000, the meeting point is guaranteed within at most $+ \approx 99,500$ steps of the fast pointer's traversal (bounded), and phase 2 takes exactly $= 99,000$ additional steps — both phases remain strictly $O(n)$ overall.

Corner Case

Single self-looping node [5] where node 5 points to itself: slow and fast both start at node 5; after one iteration, `slow = node5.next = node5`, `fast = node5.next.next = node5` → they meet immediately ($\mu=0$, $\lambda=1$) — correctly detected as a cycle.

SECTION 19 — Advanced Concepts

- **Cycle length calculation:** once a meeting point is found, keep one pointer fixed and advance the other one step at a time, counting steps until it returns to the fixed pointer — this count is exactly .
 - **Kth node in the cycle:** combine the entry-point result with a modular offset $((\text{entryIndex} + k) \bmod \text{cycleLength})$ to jump directly to any node within the cycle.
 - **Functional graph generalization:** any deterministic function $f: S \rightarrow S$ over a finite set S (like digit-square-sum for Happy Number) forms a “linked list” where node x points to $f(x)$ — Floyd’s applies directly, revealing that “does this sequence eventually cycle/terminate” problems are secretly linked-list cycle problems.
 - **Interview hack:** if you find yourself wanting a hash set “just to be safe,” pause and ask whether $O(1)$ space is explicitly required — if so, that’s your signal this pattern is expected.
-

SECTION 20 — Senior Engineer Insights

Staff engineers recognize Floyd’s algorithm as a specific case of a much broader principle: **any deterministic, finite-state, single-successor system must eventually cycle** (pigeonhole principle), and detecting that cycle in $O(1)$ space is possible whenever you can compare “state equality” cheaply. They connect this to real production concerns — for example, reasoning about whether a distributed retry/backoff state machine could loop forever, or whether a configuration-resolution chain (config A includes config B includes config A) has circular dependencies — using the exact same mental model, just applied outside of literal linked lists. Interviewers evaluate whether you can make this leap: from “linked list trick” to “general finite-state cycle detection principle.”

SECTION 21 — Cheat Sheet

PATTERN: Fast & Slow Pointers (Floyd’s Cycle Detection)

RECOGNIZE: chain/sequence traversal + cycle/middle/repeat detection + $O(1)$ space requirement

TEMPLATE:

```
slow = fast = head
while fast and fast.next:
    slow = slow.next; fast = fast.next.next
    if slow == fast: cycle detected → reset one ptr to head, advance both at speed 1 to find entry
```

COMPLEXITY: $O(n)$ time, $O(1)$ space

KEY PROOF: meeting guaranteed by pigeonhole/modular arithmetic on cycle length ; entry point found via

WATCH FOR: null checks on both fast and fast.next, self-loops, entry point meeting point

DOESN’T APPLY WHEN: multi-successor structures (trees/graphs), need full visited history

SECTION 22 — Revision Notes (5-Minute Review)

- Floyd’s = two pointers, different speeds, on a single-successor chain.
- Meeting = cycle exists (pigeonhole on cycle length).

- Entry point $\neq$ meeting point — requires phase 2 (reset one pointer to head, equal speed).
- Fast pointer at 2x speed also gives midpoint in one pass, no cycle needed.
- $O(1)$ space is the entire value proposition vs. hash-set $O(n)$ space alternative.
- Generalizes beyond literal linked lists to any deterministic function sequence (Happy Number, Find the Duplicate).
- Doesn't apply to branching structures (trees/general graphs) — use DFS coloring or Union-Find there.

SECTION 23 — Practice Roadmap

Stage	Focus	Recommended LeetCode Problems
Beginner	Basic detection and midpoint	Linked List Cycle (141), Middle of the Linked List (876)
Intermediate	Entry point, offset-based variants	Linked List Cycle II (142), Remove Nth Node From End of List (19)
Advanced	Multi-phase combinations	Palindrome Linked List (234), Reorder List (143), Sort List (148)
Expert	Non-list functional cycle detection	Happy Number (202), Find the Duplicate Number (287), Circular Array Loop (457)

SECTION 24 — Memory Tricks

- **Mnemonic:** “Tortoise and Hare” — slow tortoise, fast hare, hare laps tortoise if the track loops.
 - **Visualization:** Two runners on a **circular track** — picture the exact lap-difference math as “gap closes by 1 unit per iteration once both are on the loop.”
 - **Recognition shortcut:** “Detect cycle,” “find middle,” “ $O(1)$ space,” “linked list” together in a problem statement $\rightarrow$ Fast & Slow, no further analysis needed.
 - **Rhyme:** “Slow by one, fast by two — if they meet, the loop is true.”
-

SECTION 25 — Final Summary

Fast & Slow Pointers (Floyd's Cycle Detection) achieves $O(1)$ -space cycle and midpoint detection over chain-like structures by exploiting a simple but powerful arithmetic fact: **a faster pointer moving through a bounded, looping state space must eventually coincide with a slower one.** The single most important thing to remember forever: **the meeting point tells you a cycle exists, but finding where the cycle actually begins requires a second, distinct phase — reset one pointer to the head and advance both at equal speed, and they will provably meet at the entry point.** This pattern is a specific, elegant instance of a much broader idea — any deterministic, finite-state system eventually cycles — which is why it resurfaces far beyond linked lists, in sequence-generation problems, dependency-resolution chains, and state-machine analysis.